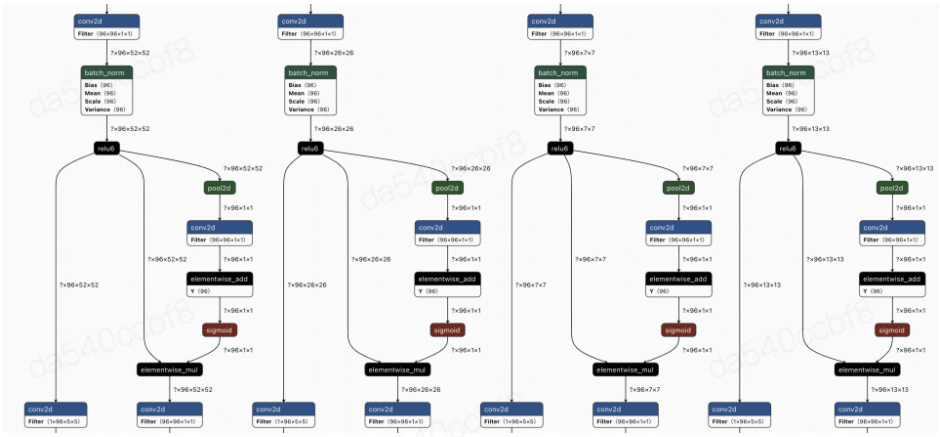
Gemv边界处理问题记录

目录

背景

梳理端侧小模型常见网络结构时，picodet模型conv1x1发现有如下结构：input shape为1x1，这会进入gemm的gemv处理分支，即M==1或N==1的情况。



在用单测评估gemv的性能时，发现basic_test选项默认关闭，仅测试单一case。

```
45  DEFINE_bool(basic_test, false, "do all tests");
```

```
206  TEST(TestLiteGemvFP16, gemv_fp16) {
207    if (FLAGS_basic_test) {
```

在conv1x1s1的单测中，同样没有对cout=1，即m=1的情况进行测试。

BenchmarkUtils > Work > Paddle-Lite > lite > tests > math > conv_fp16_compute_test.cc > TEST

```
476  #if 1  /// conv1x1s1
477  TEST(TestConv1x1s1, test_conv1x1s1) {
478    if (FLAGS_basic_test) {
479      for (auto& cin : {1, 3, 8, 11, 32}) {
480        for (auto& cout :
481             {5, 16, 37}) {  // m=1 gemv_trans run one case is ok, but run more
482                             // case in successive has diff.
483          for (auto& g : {1, 2}) {
```

```
432 -       for (auto& cout : {1, 5, 16, 37}) {
```

```
432 +       for (auto& cout :
433 +                {5, 16, 37}) {  // m=1 gemv_trans run one case is ok, but run more
434 +                                // case in successive has diff.
```

现象

```
for (auto& m : {3, 8, 397}) {
  for (auto& n : {3, 13, 16, 789}) {
    for (auto& tra : {false}) {
      for (auto& has_bias : {false, true}) {
        for (auto& flag_act : {0, 1}) {
          for (auto& th : {1, 2, 4}) {
            float six = 6.f;
            float alpha = 8.88f;
            auto flag = test_sgemv_fp16(tra,
                                        m,
                                        n,
                                        has_bias,
                                        flag_act,
                                        FLAGS_power_mode,
                                        th,
                                        six,
                                        alpha);
            if (flag) {
              VLOG(4) << "test m = " << m << ", n=" << n
                      << ", bias: " << (has_bias ? "true" : "false")
                      << ",  flag_act: " << (flag_act)
                      << ", trans A: " << (tra ? "true" : "false")
                      << " passed\n";
            } else {
              LOG(FATAL) << "test m = " << m << ", n=" << n
                         << ", bias: " << (has_bias ? "true" : "false")
                         << ",  flag_act: " << (flag_act)
                         << ", trans A: " << (tra ? "true" : "false")
                         << " failed\n";
```

打开basic_test选项后，出现数据精度对齐问题，且case多发生在m（不为8的整数倍）&& n（不为16的整数倍）。

```
gemv_fp16_compute_test.cc:193 test_sgemv_fp16] fp16 gemm M: 397, N: 789, bias: false, flag_act: 1, trans A: false, threads: 1, power_mode: 3, i: 390 failed!!
```

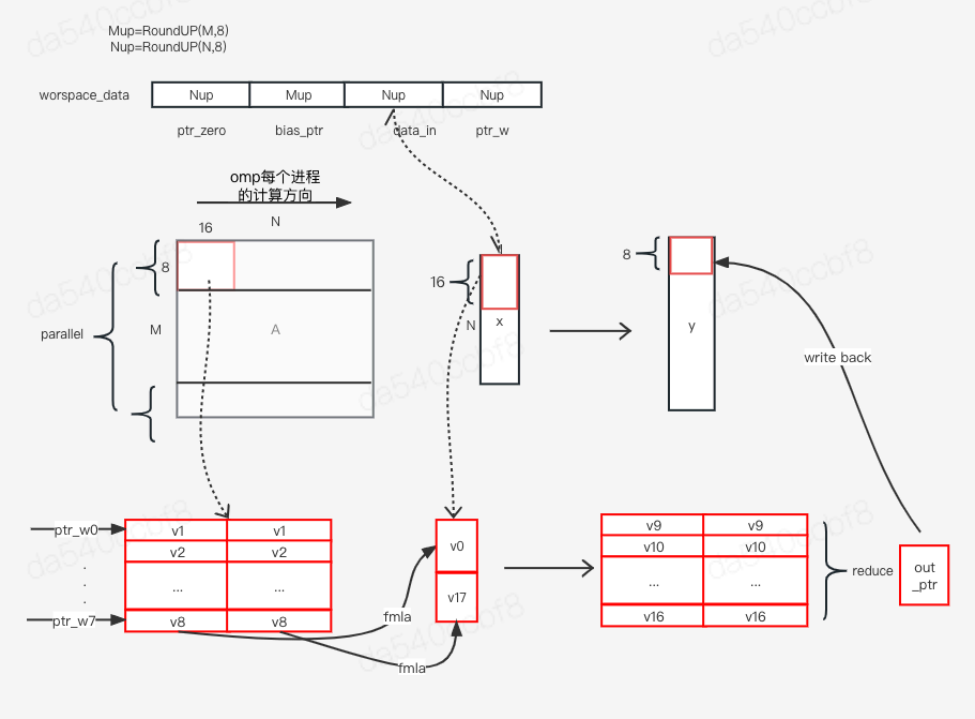
```
fp16 gemm M: 397, N: 789, bias: false, flag_act: 0, trans A: false, threads: 4, power_mode: 3, i: 140 failed!!
```

```
fp16 gemm M: 397, N: 789, bias: false, flag_act: 0, trans A: false, threads: 1, power_mode: 3, i: 301 failed!!
```

当前实现方案

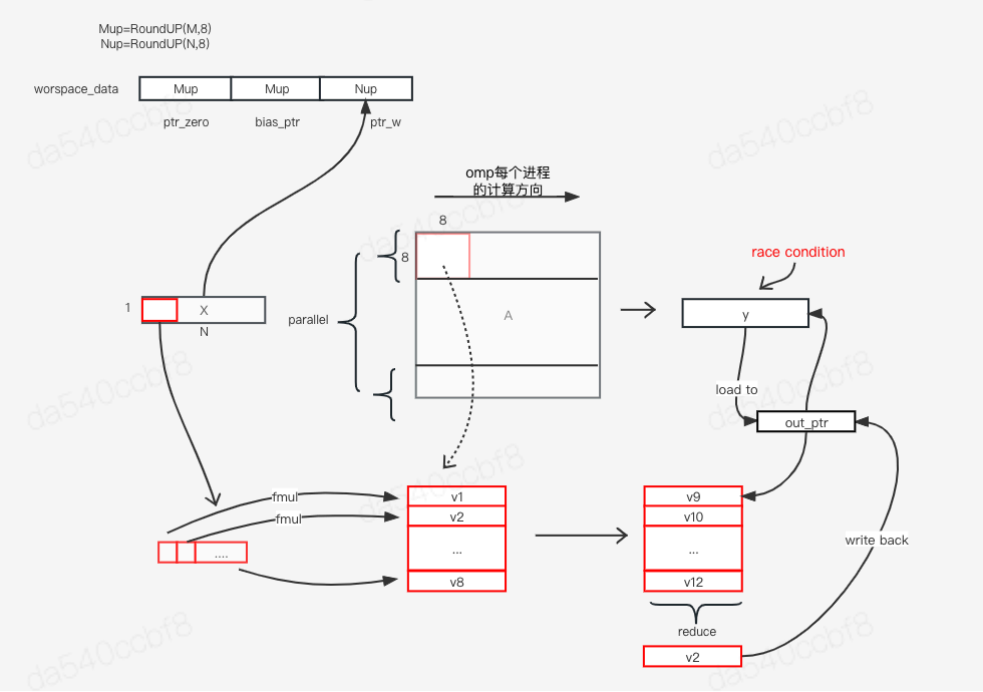
gemv_fp16_no_trans

具体计算过程如下所示：



gemv_fp16_trans

具体计算过程如下所示：



对结果矩阵y的读写操作存在race condition情况，需要重构代码。

TODO

- [x] 单测精度对齐问题
- [x] gemv_fp16_trans bug修复